DevOps & Cloud Assignments

Solutions for Assignments 2 to 10

Assignment 2: Docker Basics

Assignment 3: Volumes and Dockerfiles

Assignment 4: Docker Compose and Minikube

Assignment 5: Docker Swarm

Assignment 6: Kubernetes

Assignment 7: Jenkins CI/CD Pipeline

Assignment 8: AWS (EC2, S3, VPC, IAM)

Assignment 9: AWS Lambda + DynamoDB + S3

Assignment 10: Terraform + Ansible

Replace placeholders (PRN, name, usernames, IPs, key names) with your own details before running.

Assignment 2: Docker Basics

1: Pull and Run Image

```
docker pull nginx                      # 1. Pull image from Docker Hub
docker run -d --name my_nginx nginx    # 2. Run container in background
docker ps                              # 3. Check running containers
docker stop my_nginx                   # 4. Stop the container
docker rm my_nginx                     # 5. Remove the container
```

2: Image Commands Practice

```
docker pull alpine          # 1. Pull alpine
docker images               # 2. List local images
docker inspect alpine       # 3. Inspect image (layers, env, cmd...)
docker rmi alpine           # 4. Remove alpine image
```

5. What happens if you try to remove a running container's image?

```
docker pull alpine
docker run -d --name test_alpine alpine sleep 1000
docker rmi alpine
```

Docker refuses with an error like:

```
Error response from daemon: conflict: unable to remove repository reference "alpine"
(must force) - container <id> is using its referenced image <image-id>
```

Explanation: A container is created from the image's read-only layers, so it depends on them while it exists. Docker blocks the removal to protect the container. Even `docker rmi -f alpine` only removes the tag and does not delete the layers in use. To remove the image properly, first stop and remove the container, then remove the image:

```
docker stop test_alpine && docker rm test_alpine
docker rmi alpine
```

3: Container Commands

```
docker run -it --name ubuntu_test ubuntu bash   # 1. Interactive mode
ls                                              # 2. Run inside container
pwd
exit                                            # 3. Exit (container stops)

docker ps -a                                    # 4. List all containers
docker start -ai ubuntu_test                    # 5. Restart stopped container
# or: docker restart ubuntu_test
```

4: Naming and Renaming

```
docker run -d --name web_server nginx   # 1. Run with a name
docker ps                               # 2. Verify the name (NAMES column)
docker stop web_server                  # 3. Stop
docker rename web_server my_server      # 4. Rename
docker ps -a                            #    verify new name
docker rm my_server                     # 5. Remove
```

5: Tagging Docker Image

```
docker pull nginx                  # 1. Pull
docker tag nginx mynginx:v1        # 2. Tag as mynginx:v1
docker tag nginx mynginx:latest    # 3. Tag as mynginx:latest
docker images                      # 4. Verify tags
docker rmi mynginx:v1              # 5. Remove one tag
docker images
```

Observation: All tags share the same IMAGE ID. Removing `mynginx:v1` prints `Untagged: mynginx:v1` and deletes only that tag. The image data stays because other tags still point to it. Layers are removed only when

the last tag is removed.

6: Push Image to Docker Hub

```
docker login                                    # 1. Enter username & password/token
docker tag nginx <your_username>/mynginx:v1     # 2. Tag with your username
docker push <your_username>/mynginx:v1          # 3. Push
docker logout                                   # 4. Logout
```

5. Verify: Open https://hub.docker.com, log in, and open Repositories. You should see `<your_username>/mynginx` with tag `v1`. Replace `<your_username>` with your real (lowercase) Docker Hub username.

7: Basic Networking and Port Mapping

```
docker run -d --name web -p 8080:80 nginx   # 1. Port mapping host:container
# 2. Browser -> http://localhost:8080 (shows "Welcome to nginx!")
docker network ls                           # 3. List networks
docker network inspect bridge               # 4. Inspect default bridge network
```

In the inspect output, look at `Subnet` (usually 172.17.0.0/16), `Gateway`, and the `Containers` section listing your container with its IP.

5. What port mapping does: A container runs in its own isolated network with a private IP, so it cannot be reached directly from outside. Port mapping (`-p HOST_PORT:CONTAINER_PORT`) forwards traffic arriving on a port of the host to a port inside the container. With `-p 8080:80`, a request to `localhost:8080` reaches port 80 of the nginx container. It also lets several containers that use the same internal port run side by side on different host ports.

Cleanup:

```
docker stop web && docker rm web
```

Assignment 3: Volumes and Dockerfiles

Q1. Docker Volume Persistence

```
docker volume create data_vol
docker run -it --name c1 -v data_vol:/data ubuntu bash
# inside container:
echo "Hello persistent" > /data/file.txt
ls /data
exit

docker rm c1

docker run -it --name c2 -v data_vol:/data ubuntu bash
cat /data/file.txt        # file still exists
exit
```

Result: The file still exists. The volume is stored on the host and lives independently of any container's lifecycle.

Q2. Dockerfile: Python Application

app.py

```
print("Hello from Python Docker container!")
```

Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY app.py .
CMD ["python", "app.py"]

docker build -t python-app .
docker run --rm python-app
```

Q3. Dockerfile: Java Application

Hello.java

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello from Java Docker container!");
    }
}
```

Dockerfile

```
FROM eclipse-temurin:17-jdk
WORKDIR /app
COPY Hello.java .
RUN javac Hello.java
CMD ["java", "Hello"]

docker build -t java-app .
docker run --rm java-app
```

Q4. Docker Volume with MySQL

```
docker volume create mysql_data
docker run -d --name mysql_db \
  -e MYSQL_ROOT_PASSWORD=root123 \
  -v mysql_data:/var/lib/mysql \
  mysql:8

# wait ~20-30 seconds for MySQL to initialize, then:
docker exec -it mysql_db mysql -uroot -proot123
```

Inside MySQL:

```
CREATE DATABASE school;
```

```
USE school;
CREATE TABLE students (id INT PRIMARY KEY, name VARCHAR(50));
INSERT INTO students VALUES (1, 'Amit'), (2, 'Priya');
SELECT * FROM students;
EXIT;
```

Restart and verify:

```
docker restart mysql_db
docker exec -it mysql_db mysql -uroot -proot123 -e "SELECT * FROM school.students;"
```

Result: The data persists because MySQL's data directory is stored in the named volume.

Q5. Dockerfile: Nginx Web Server

index.html

```
<!DOCTYPE html>
<html>
<body>
  <h1>Hello from my custom Nginx page!</h1>
</body>
</html>
```

Dockerfile

```
FROM nginx:latest
COPY index.html /usr/share/nginx/html/index.html

docker build -t custom-nginx .
docker run -d --name my_site -p 8080:80 custom-nginx
```

Open http://localhost:8080 in the browser.

Q6. Bind Mount vs Volume

```
mkdir ~/host_dir
echo "File from host" > ~/host_dir/host.txt

# Bind mount
docker run -it --rm -v ~/host_dir:/data ubuntu bash
cat /data/host.txt
echo "added in container" >> /data/host.txt
exit
cat ~/host_dir/host.txt     # change visible on host

# Named volume (for comparison)
docker run -it --rm -v data_vol:/data ubuntu bash
```

Difference:

Aspect	Bind Mount	Docker Volume
Location	Any host path you choose	Managed by Docker (/var/lib/docker/volumes/)
Created by	You, manually	docker volume create or automatically
Host dependency	Depends on host directory structure	Independent, more portable
Best for	Development (live code editing)	Production data (databases)
Management	Manual	Docker CLI (volume ls/inspect/rm)
Syntax	-v /host/path:/container/path	-v volume_name:/container/path

Q7. Dockerfile: Shell Script Execution

script.sh

```
#!/bin/sh
echo "Hello from shell script inside Docker!"
date
```

Dockerfile

```
FROM alpine
WORKDIR /app
COPY script.sh .
RUN chmod +x script.sh
CMD ["./script.sh"]

docker build -t shell-app .
docker run --rm shell-app
```

Q8. Shared Volume Between Containers

```
docker volume create shared_vol

# Container 1 writes data
docker run --rm --name writer -v shared_vol:/shared ubuntu \
  bash -c "echo 'Data from container 1' > /shared/data.txt"

# Container 2 reads the same data
docker run --rm --name reader -v shared_vol:/shared ubuntu \
  cat /shared/data.txt
```

Output: `Data from container 1`. Both containers access the same volume, so they share the data.

To show both running at once:

```
docker run -dit --name c1 -v shared_vol:/shared ubuntu bash
docker run -dit --name c2 -v shared_vol:/shared ubuntu bash
docker exec c1 bash -c "echo 'hello from c1' > /shared/msg.txt"
docker exec c2 cat /shared/msg.txt
```

Assignment 4: Docker Compose and Minikube

Q1. Docker Compose: Nginx Static Website

```
mkdir -p website && cd website
echo "<h1>Hello from Docker Compose</h1>" > index.html
cd ..
```

docker-compose.yml

```
services:
  web:
    image: nginx
    ports:
      - "8080:80"
    volumes:
      - ./website:/usr/share/nginx/html

docker compose up -d
docker compose down      # when finished
```

Open http://localhost:8080 and you should see **Hello from Docker Compose**.

Q2. Docker Compose: Python Script Container

app.py

```
print("Hello from Python Container")
```

Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY app.py .
CMD ["python", "app.py"]
```

docker-compose.yml

```
services:
  python-app:
    build: .

docker compose up --build
docker compose ps -a      # shows Exited (0)
docker compose down
```

The container prints the message and then exits with code 0 because the script finishes.

Q3. Docker Compose: MySQL with Volume

docker-compose.yml

```
services:
  db:
    image: mysql:8
    container_name: mysql_db
    environment:
      MYSQL_ROOT_PASSWORD: root123
      MYSQL_DATABASE: mydb
    volumes:
      - mysql_data:/var/lib/mysql

volumes:
  mysql_data:

docker compose up -d
# wait ~20-30 seconds
docker exec -it mysql_db mysql -uroot -proot123
```

Inside MySQL:

```
USE mydb;
CREATE TABLE test (id INT, name VARCHAR(50));
INSERT INTO test VALUES (1, 'Amit');
SELECT * FROM test;
EXIT;
```

Restart and verify:

```
docker compose restart
docker exec -it mysql_db mysql -uroot -proot123 -e "SELECT * FROM mydb.test;"
```

The data is still there because it is stored in the named volume `mysql_data`. Use `docker compose down` (without -v) to keep the volume.

Q4. One Volume with 3 Nginx Containers

All three containers mount the same volume at /usr/share/nginx/html, so they would normally serve the same file. Here each container writes its own page into the shared volume and is configured to serve a different file from it.

docker-compose.yml

```
services:
  nginx1:
    image: nginx
    container_name: nginx1
    ports:
      - "8081:80"
    volumes:
      - shared-data:/usr/share/nginx/html
    command: >
      sh -c "echo 'Hello from Container 1' > /usr/share/nginx/html/page1.html &&
             sed -i 's#index  index.html index.htm;#index page1.html;#'
                 /etc/nginx/conf.d/default.conf &&
             nginx -g 'daemon off;'"

  nginx2:
    image: nginx
    container_name: nginx2
    ports:
      - "8082:80"
    volumes:
      - shared-data:/usr/share/nginx/html
    command: >
      sh -c "echo 'Hello from Container 2' > /usr/share/nginx/html/page2.html &&
             sed -i 's#index  index.html index.htm;#index page2.html;#'
                 /etc/nginx/conf.d/default.conf &&
             nginx -g 'daemon off;'"

  nginx3:
    image: nginx
    container_name: nginx3
    ports:
      - "8083:80"
    volumes:
      - shared-data:/usr/share/nginx/html
    command: >
      sh -c "echo 'Hello from Container 3' > /usr/share/nginx/html/page3.html &&
             sed -i 's#index  index.html index.htm;#index page3.html;#'
                 /etc/nginx/conf.d/default.conf &&
             nginx -g 'daemon off;'"

volumes:
  shared-data:

docker compose up -d
docker compose ps
```

Verify in the browser:

- http://localhost:8081 shows Hello from Container 1

- http://localhost:8082 shows Hello from Container 2

- http://localhost:8083 shows Hello from Container 3

Check the volume is shared:

```
docker exec nginx1 ls /usr/share/nginx/html
docker compose down -v
```

Q5. Kubernetes Installation Steps (Minikube)

Prerequisites: 2+ CPUs, 2 GB+ RAM, 20 GB disk, internet, and a container or VM manager (Docker is easiest).

Step 1: Update the system and install Docker

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y docker.io curl
sudo usermod -aG docker $USER
newgrp docker
```

Step 2: Install kubectl

```
curl -LO "https://dl.k8s.io/release/$(curl -L -s
    https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
chmod +x kubectl
sudo mv kubectl /usr/local/bin/
kubectl version --client
```

Step 3: Install Minikube

```
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube
minikube version
```

Step 4: Start the cluster

```
minikube start --driver=docker
```

Step 5: Verify

```
minikube status
kubectl get nodes          # node should show STATUS = Ready
kubectl cluster-info
```

Step 6: Test with a sample app

```
kubectl create deployment hello --image=nginx
kubectl expose deployment hello --type=NodePort --port=80
minikube service hello --url
```

Useful commands

```
minikube stop       # stop the cluster
minikube delete     # delete the cluster
minikube dashboard  # open the web UI
```

On Windows (brief): install Docker Desktop (or enable Hyper-V), install kubectl and Minikube (choco install kubernetes-cli minikube), run `minikube start --driver=docker`, then verify with `kubectl get nodes`.

Assignment 5: Docker Swarm

Q1. Nginx Service

```
docker swarm init
# multiple IPs: docker swarm init --advertise-addr <your-ip>

docker service create \
  --name nginx-service \
  --replicas 2 \
  -p 8080:80 \
  nginx

docker service ls
docker service ps nginx-service
```

Open http://localhost:8080 and you should see "Welcome to nginx!". The routing mesh serves the port on every node.

Q2. Python Worker Service

app.py

```
import time

while True:
    print("Running in Swarm...", flush=True)
    time.sleep(5)
```

Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY app.py .
CMD ["python", "app.py"]

docker build -t python-worker .

docker service create \
  --name python-service \
  --replicas 3 \
  python-worker
```

On a multi-node swarm the image must be available on all nodes (push to Docker Hub or a registry). On a single node the local image is enough.

Check logs from different containers:

```
docker service logs python-service       # logs from all replicas
docker ps                                # find individual container names
docker logs <container_id>               # logs of one container
```

Verify task distribution:

```
docker service ps python-service
```

This shows 3 tasks (python-service.1, .2, .3), the node each runs on, and its state.

Q3. Scaling Service

```
docker service create --name nginx-service --replicas 2 -p 8080:80 nginx
# (skip if nginx-service from Q1 is still running)

docker service scale nginx-service=5
docker service ls
docker service ps nginx-service

docker service scale nginx-service=1
docker service ps nginx-service
docker ps
```

Observation: Scaling up creates 3 new containers. Scaling down to 1 shuts down 4 tasks; their containers are stopped and removed (they may briefly show as Shutdown in `docker service ps`) and one replica keeps running. The service stays available throughout.

Q4. Rolling Update

```
docker service create \
  --name rolling-service \
  --replicas 4 \
  -p 8081:80 \
  nginx:1.25

docker service update \
  --image nginx:latest \
  --update-parallelism 1 \
  --update-delay 10s \
  rolling-service
```

`--update-parallelism 1` updates one container at a time. `--update-delay 10s` waits 10 seconds between updates.

Verify update status:

```
docker service ps rolling-service
docker service inspect --pretty rolling-service    # UpdateStatus: completed
docker service ls
```

Roll back if needed:

```
docker service rollback rolling-service
```

Q5. Remove Services and Leave Swarm

```
docker service ls
docker service rm nginx-service
docker service rm python-service
docker service rm rolling-service     # extra, from Q4
docker service ls                     # verify: empty list
docker swarm leave --force            # --force needed on a manager
```

Q6. Usage of Docker Swarm

Docker Swarm is Docker's built-in clustering and orchestration tool. It turns a group of Docker hosts into a single virtual host.

1. **Container orchestration:** Swarm manages the lifecycle of containers across a cluster: placement, startup, and restarts. Managers schedule, workers run containers.

2. **Service deployment:** You declare a desired state (image, replicas, ports) with `docker service create` or `docker stack deploy`. Swarm keeps the actual state equal to it.

3. **Scaling containers:** `docker service scale web=10` adds or removes replicas instantly, spread across nodes.

4. **Load balancing:** The routing mesh exposes a published port on every node and distributes requests across replicas. Internal DNS-based load balancing handles service-to-service traffic.

5. **Rolling updates:** Images update with minimal downtime, with controlled parallelism and delay, and can be rolled back.

6. **High availability:** If a container or node fails, Swarm reschedules tasks on healthy nodes. Multiple managers use the Raft consensus algorithm, so the cluster survives manager failures (a majority must stay available).

7. **Multi-node container management:** Many machines are managed as one cluster, with encrypted overlay networks, secrets, and mutual TLS between nodes, all through the Docker CLI.

Q7. Limitations of Docker Swarm

1. **Less powerful than Kubernetes:** No CRDs or operators, less fine-grained scheduling (affinity, taints, tolerations), fewer deployment strategies.

2. **Smaller ecosystem:** Far fewer third-party tools, packaging systems, plugins, and community resources.

3. **Limited auto-scaling support:** No built-in autoscaler. Scaling is manual unless external tooling is added. Kubernetes has HPA/VPA.

4. **Limited networking features:** Simpler networking, no network policies for fine-grained traffic control, no ingress controller concept.

5. **Limited storage management:** No equivalent of PersistentVolumes/StorageClasses or dynamic provisioning. Stateful workloads need external volume plugins.

6. **Not widely used in large production systems:** Most large organizations and cloud providers standardized on Kubernetes, so Swarm has less managed-service support and slower development.

7. **Fewer monitoring and security integrations:** Native observability, RBAC, and policy tooling is limited compared to Kubernetes' mature ecosystem.

Assignment 6: Kubernetes

Assumes Minikube is running (`minikube start`) and `kubectl` works.

Q1. Run Apache Pod

```
kubectl run apache-pod --image=httpd
kubectl get pods
kubectl describe pod apache-pod
```

Wait until STATUS shows Running.

Q2. Apache Deployment

```
kubectl create deployment apache-deployment --image=httpd
kubectl get deployments
kubectl get pods
```

Q3. Increase and Decrease Replicas

```
kubectl scale deployment apache-deployment --replicas=4
kubectl get rs
kubectl get pods              # 4 pods

kubectl scale deployment apache-deployment --replicas=2
kubectl get rs
kubectl get pods              # 2 pods (extras Terminating, then gone)
```

The ReplicaSet is created automatically by the Deployment, and its DESIRED/CURRENT/READY columns change with scaling.

Q4. Use YAML File

apache-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: apache-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: apache
  template:
    metadata:
      labels:
        app: apache
    spec:
      containers:
      - name: apache
        image: httpd
        ports:
        - containerPort: 80
```

The Pod and ReplicaSet are created automatically by the Deployment. For a standalone Pod:

apache-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: apache-pod-yaml
  labels:
    app: apache-standalone
spec:
  containers:
  - name: apache
    image: httpd
```

```
    ports:
    - containerPort: 80
```

```
kubectl delete deployment apache-deployment    # remove the one from Q2/Q3 first
kubectl apply -f apache-deployment.yaml
kubectl apply -f apache-pod.yaml
kubectl get all
```

Q5. ClusterIP Service

```
kubectl expose deployment apache-deployment \
  --name=apache-clusterip --type=ClusterIP --port=80
kubectl get svc
```

Or YAML:

```
apiVersion: v1
kind: Service
metadata:
  name: apache-clusterip
spec:
  type: ClusterIP
  selector:
    app: apache
  ports:
  - port: 80
    targetPort: 80
```

Test inside the cluster:

```
kubectl run test-pod --rm -it --image=busybox -- sh
wget -qO- http://apache-clusterip
exit
```

Output: `<html><body><h1>It works!</h1></body></html>`

Q6. NodePort Service

```
kubectl expose deployment apache-deployment \
  --name=apache-nodeport --type=NodePort --port=80
kubectl get svc apache-nodeport      # note the port, e.g. 80:31234/TCP
```

Or YAML:

```
apiVersion: v1
kind: Service
metadata:
  name: apache-nodeport
spec:
  type: NodePort
  selector:
    app: apache
  ports:
  - port: 80
    targetPort: 80
    nodePort: 30080
```

Access via http://<node-ip>:<node-port>:

```
minikube ip
minikube service apache-nodeport --url
```

Q7. LoadBalancer Service

```
kubectl expose deployment apache-deployment \
  --name=apache-loadbalancer --type=LoadBalancer --port=80
kubectl get svc
```

On Minikube, EXTERNAL-IP shows `<pending>` until you run this in a separate terminal and keep it running:

```
minikube tunnel
```

Then `kubectl get svc` shows an external IP. On a cloud cluster (EKS/GKE/AKS) the provider assigns it automatically.

Q8. Flask Application Deployment

app.py

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "<h1>PRN: 123456789</h1><h2>Name: Your Name</h2>"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

requirements.txt

```
flask
```

Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

Build inside Minikube's Docker environment so the cluster sees the image without a registry:

```
eval $(minikube docker-env)
docker build -t flask-app:v1 .
```

flask-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: flask-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: flask
  template:
    metadata:
      labels:
        app: flask
    spec:
      containers:
      - name: flask
        image: flask-app:v1
        imagePullPolicy: Never      # use the locally built image
        ports:
        - containerPort: 5000
---
apiVersion: v1
kind: Service
metadata:
  name: flask-service
spec:
  type: NodePort
  selector:
    app: flask
  ports:
  - port: 80
    targetPort: 5000
    nodePort: 30007
```

```
kubectl apply -f flask-deployment.yaml
kubectl get pods
kubectl get svc
minikube service flask-service --url     # open in browser
```

If you pushed to Docker Hub instead, remove `imagePullPolicy: Never` and use the Hub image name.

Q9. Rolling Update Flask Application

1. Modify app.py

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return ("<h1>PRN: 123456789</h1>"
            "<h2>Name: Your Name</h2>"
            "<h3>Course: DevOps / Cloud Computing</h3>")

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

2. Build the new image version

```
eval $(minikube docker-env)
docker build -t flask-app:v2 .
```

3. Update the Deployment (`flask` is the container name in the YAML)

```
kubectl set image deployment/flask-deployment flask=flask-app:v2
```

4. Verify the rolling update

```
kubectl rollout status deployment/flask-deployment
kubectl get pods
kubectl rollout history deployment/flask-deployment
```

5. Refresh the `minikube service flask-service --url` page. It now shows PRN, name, and course.

Rollback if needed:

```
kubectl rollout undo deployment/flask-deployment
```

Replace the PRN, name, and course with your own details.

Assignment 7: Jenkins CI/CD Pipeline

Assumes Jenkins is installed with the Git, Pipeline, Docker Pipeline, and Credentials plugins. The `jenkins` user must be able to run Docker (`sudo usermod -aG docker jenkins && sudo systemctl restart jenkins`) and `kubectl` (with a working kubeconfig). Replace `<github_user>` and `<dockerhub_user>` with your own usernames.

1. Java Application with Jenkins Pipeline

HelloWorld.java

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello World from Jenkins Pipeline!");
    }
}
```

Push to GitHub:

```
git init
git add HelloWorld.java
git commit -m "Add Hello World app"
git branch -M main
git remote add origin https://github.com/<github_user>/java-hello.git
git push -u origin main
```

Create the job: Jenkins -> New Item -> name `java-pipeline` -> Pipeline -> OK. Under Pipeline choose "Pipeline script" and paste:

```
pipeline {
    agent any
    stages {
        stage('Pull Code') {
            steps {
                git branch: 'main', url: 'https://github.com/<github_user>/java-hello.git'
            }
        }
        stage('Compile') {
            steps {
                sh 'javac HelloWorld.java'
            }
        }
        stage('Run') {
            steps {
                sh 'java HelloWorld'
            }
        }
    }
}
```

Verify: Build Now -> open the build -> Console Output shows `Hello World from Jenkins Pipeline!` and `Finished: SUCCESS`. A JDK must be installed on the Jenkins server.

2. Docker Image Creation using Jenkins

Dockerfile (add to the repo and push)

```
FROM eclipse-temurin:17-jdk
WORKDIR /app
COPY HelloWorld.java .
RUN javac HelloWorld.java
CMD ["java", "HelloWorld"]
```

Updated pipeline

```
pipeline {
    agent any
    environment {
        IMAGE_NAME = 'java-hello'
```

```
    }
    stages {
        stage('Pull Code') {
            steps {
                git branch: 'main', url: 'https://github.com/<github_user>/java-hello.git'
            }
        }
        stage('Compile') {
            steps { sh 'javac HelloWorld.java' }
        }
        stage('Run') {
            steps { sh 'java HelloWorld' }
        }
        stage('Build Docker Image') {
            steps { sh 'docker build -t $IMAGE_NAME:latest .' }
        }
        stage('Run Container') {
            steps { sh 'docker run --rm $IMAGE_NAME:latest' }
        }
    }
}
```

Verify: The `Run Container` stage output shows `Hello World from Jenkins Pipeline!`. Also check `docker images` on the Jenkins server.

3. Push Docker Image to DockerHub

Configure credentials: Jenkins -> Manage Jenkins -> Credentials -> System -> Global credentials -> Add Credentials. Kind: Username with password. Username: your Docker Hub username. Password: Docker Hub password or access token (recommended). ID: `dockerhub-creds`.

Extended pipeline

```
pipeline {
    agent any
    environment {
        IMAGE_NAME = 'java-hello'
        DOCKERHUB_USER = '<dockerhub_user>'
        IMAGE_TAG = "${BUILD_NUMBER}"
    }
    stages {
        stage('Pull Code') {
            steps {
                git branch: 'main', url: 'https://github.com/<github_user>/java-hello.git'
            }
        }
        stage('Compile & Run') {
            steps {
                sh 'javac HelloWorld.java'
                sh 'java HelloWorld'
            }
        }
        stage('Build Docker Image') {
            steps { sh 'docker build -t $IMAGE_NAME:latest .' }
        }
        stage('Run Container') {
            steps { sh 'docker run --rm $IMAGE_NAME:latest' }
        }
        stage('Tag Image') {
            steps {
                sh '''
                  docker tag $IMAGE_NAME:latest $DOCKERHUB_USER/$IMAGE_NAME:latest
                  docker tag $IMAGE_NAME:latest $DOCKERHUB_USER/$IMAGE_NAME:$IMAGE_TAG
                '''
            }
        }
        stage('Push to DockerHub') {
            steps {
                withCredentials([usernamePassword(credentialsId: 'dockerhub-creds',
                                                  usernameVariable: 'DH_USER',
                                                  passwordVariable: 'DH_PASS')]) {
                    sh '''
```

```
                    echo "$DH_PASS" | docker login -u "$DH_USER" --password-stdin
                    docker push $DOCKERHUB_USER/$IMAGE_NAME:latest
                    docker push $DOCKERHUB_USER/$IMAGE_NAME:$IMAGE_TAG
                    docker logout
                '''
            }
        }
    }
}
```

Verify: On hub.docker.com -> Repositories, `<dockerhub_user>/java-hello` shows the tags `latest` and the build number.

4. Deploy Application to Kubernetes using Jenkins

k8s-deployment.yaml (add to the repo)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: java-hello-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: java-hello
  template:
    metadata:
      labels:
        app: java-hello
    spec:
      containers:
      - name: java-hello
        image: <dockerhub_user>/java-hello:latest
        imagePullPolicy: Always
        command: ["sh", "-c", "java HelloWorld && sleep 3600"]
---
apiVersion: v1
kind: Service
metadata:
  name: java-hello-service
spec:
  type: NodePort
  selector:
    app: java-hello
  ports:
  - port: 80
    targetPort: 8080
```

The Java app prints and exits, so without the `command` line the pod would restart in a loop (CrashLoopBackOff). The `command` keeps it running after printing so the pod stays in Running state.

Final pipeline stage

```
        stage('Deploy to Kubernetes') {
            steps {
                sh '''
                  kubectl apply -f k8s-deployment.yaml
                  kubectl rollout restart deployment/java-hello-deployment
                  kubectl rollout status deployment/java-hello-deployment --timeout=120s
                '''
            }
        }
```

The cluster pulls the image from DockerHub because of the image name in the YAML.

Kubeconfig access for the jenkins user

```
sudo mkdir -p /var/lib/jenkins/.kube
sudo cp ~/.kube/config /var/lib/jenkins/.kube/config
sudo chown -R jenkins:jenkins /var/lib/jenkins/.kube
```

For Minikube, certificate paths inside the config must also be readable by `jenkins`.

Verify the pod is running

```
kubectl get pods
kubectl logs deployment/java-hello-deployment
kubectl get svc
```

STATUS should be Running and the logs show `Hello World from Jenkins Pipeline!`.

Assignment 8: AWS (EC2, S3, VPC, IAM)

All steps use the AWS Management Console. Pick one region (e.g. ap-south-1 Mumbai) and use it throughout.

Q1. Create EC2 Instance and Connect

1. Console -> EC2 -> Launch instance.
2. Name: `basic-ec2`
3. AMI: Ubuntu Server (22.04 or 24.04 LTS)
4. Instance type: `t3.micro`
5. Key pair: Create new key pair -> name `basic-key` -> RSA, .pem -> download and keep it safe.
6. Network settings -> Create security group: allow SSH (port 22), Source: My IP.
7. Click Launch instance.
8. Select the instance -> Connect -> EC2 Instance Connect tab -> Connect. A browser terminal opens.

Note: Instance Connect needs SSH reachable from AWS's IP range for your region. If it fails with "My IP" only, temporarily allow the EC2 Instance Connect range or use 0.0.0.0/0 for the demo.

Q2. Create S3 Bucket and Upload File

1. Console -> S3 -> Create bucket.
2. Name: `my-basic-bucket-001` (add a suffix if taken; names are globally unique).
3. For the public access test, untick "Block all public access" and acknowledge the warning. Create bucket.
4. Open the bucket -> Upload -> Add files (e.g. index.html) -> Upload.
5. Click the file -> copy the Object URL.
6. Make it public (if needed) using a bucket policy under Permissions:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": "*",
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::my-basic-bucket-001/*"
  }]
}
```

7. Paste the Object URL in the browser to view the file.

Q3. Create VPC with Public Subnet

1. Create VPC: VPC -> Your VPCs -> Create VPC -> VPC only. Name `my-vpc`, IPv4 CIDR `10.0.0.0/16`.

2. Create Subnet: VPC -> Subnets -> Create subnet -> VPC `my-vpc`. Name `public-subnet`, CIDR `10.0.1.0/24`.

3. Create Internet Gateway: VPC -> Internet gateways -> Create, name `my-igw`. Actions -> Attach to VPC -> `my-vpc`.

4. Create Route Table: VPC -> Route tables -> Create route table, name `public-rt`, VPC `my-vpc`. Routes -> Edit routes -> Add route: destination `0.0.0.0/0`, target Internet Gateway `my-igw`. Subnet associations -> Edit -> select `public-subnet`.

Also enable auto-assign public IP: Subnets -> `public-subnet` -> Actions -> Edit subnet settings -> tick "Enable auto-assign public IPv4 address".

Q4. Launch EC2 Inside Your VPC

1. EC2 -> Launch instance -> Name `vpc-ec2`.

2. AMI Ubuntu, type `t3.micro`, key pair `basic-key`.

3. Network settings -> Edit: VPC `my-vpc`, Subnet `public-subnet`, Auto-assign public IP: Enable.

4. Security group (new, e.g. `vpc-ec2-sg`): SSH (22) from My IP; HTTP (80) from `0.0.0.0/0`.

5. Launch instance.

6. Select it -> Connect -> EC2 Instance Connect -> Connect.

Optional HTTP test:

```
sudo apt update && sudo apt install -y nginx
```

Then open http://<public-ip> in the browser.

Q5. Attach IAM Role to EC2 and Access S3

1. Create the role: IAM -> Roles -> Create role. Trusted entity: AWS service, use case EC2. Attach policy `AmazonS3FullAccess`. Role name `ec2-s3-role`.

2. Attach to the instance: EC2 -> select `vpc-ec2` -> Actions -> Security -> Modify IAM role -> choose `ec2-s3-role` -> Update IAM role.

3. Verify in the EC2 terminal:

```
sudo apt update && sudo apt install -y awscli
# if awscli is not in apt: sudo snap install aws-cli --classic
aws s3 ls
```

It lists your buckets without any access keys configured, because the instance uses the role's temporary credentials. The instance needs internet access (public subnet with IGW) to reach S3.

Q6. Create Private Subnet + EC2 (No Internet)

1. Create subnet: VPC -> Subnets -> Create subnet, VPC `my-vpc`, name `private-subnet`, CIDR `10.0.2.0/24`. Do not enable auto-assign public IP.

2. Create route table: name `private-rt`, VPC `my-vpc`. Keep only the default `local` route (no Internet Gateway route). Subnet associations -> select `private-subnet`.

3. Launch EC2: name `private-ec2`, VPC `my-vpc`, subnet `private-subnet`, auto-assign public IP: Disable, key pair `basic-key`.

4. Security group (`private-ec2-sg`): SSH (22), source `10.0.0.0/16` (the VPC CIDR).

Q7. Create Bastion Host (Jump Server)

1. Launch the bastion: name `bastion-host`, VPC `my-vpc`, subnet `public-subnet`, auto-assign public IP: Enable, key pair `basic-key`.

2. Bastion security group (`bastion-sg`): SSH (22) from My IP.

3. Modify the private EC2's security group: EC2 -> `private-ec2` -> Security tab -> click `private-ec2-sg` -> Edit inbound rules. Delete the `10.0.0.0/16` rule. Add SSH (22) with source type Custom -> select the security group `bastion-sg` (not an IP) -> Save.

Q8. Verify Private Access via Bastion

1. Connect to the bastion host (EC2 Instance Connect, or SSH from your PC).

2. Use SSH agent forwarding from your PC (safest):

```
chmod 400 basic-key.pem
ssh-add basic-key.pem
ssh -A -i basic-key.pem ubuntu@<bastion-public-ip>
```

3. From the bastion:

```
ssh ubuntu@<private-ec2-private-ip>
```

Alternatively, copy the key to the bastion (simpler, less secure; delete it afterwards):

```
# on your PC
scp -i basic-key.pem basic-key.pem ubuntu@<bastion-public-ip>:~/key.pem
# on the bastion
chmod 400 key.pem
ssh -i key.pem ubuntu@<private-ec2-private-ip>
```

Observation: The private EC2 has no public IP, so it cannot be reached directly from the internet. Its security group allows SSH only from the bastion's security group, so it is reachable only through the bastion. The prompt changes to `ubuntu@ip-10-0-2-x` once you are in.

Q9. Create Two IAM Users (Access vs No Access)

User 1: IAM -> Users -> Create user. Name `s3-user-allowed`. Tick "Provide user access to the AWS Management Console", set a custom password. Next -> Attach policies directly -> `AmazonS3FullAccess` -> Create user.

User 2: Create user, name `s3-user-denied`. Enable console access and set a password. Next -> do not attach any policy -> Create user.

Save each user's console sign-in URL (form: https://<account-id>.signin.aws.amazon.com/console).

Q10. Test S3 Access (Permission Check)

Use separate browsers or incognito windows.

Login as s3-user-allowed: open the sign-in URL, log in, go to S3, open `my-basic-bucket-001`, and Upload a file. Result: succeeds.

Login as s3-user-denied: sign in, go to S3. You see "Insufficient permissions to list buckets" or Access Denied errors. Try to upload by opening the bucket directly (https://s3.console.aws.amazon.com/s3/buckets/my-basic-bucket-001) and clicking Upload. Result: fails with Access Denied.

Conclusion: IAM users have no permissions by default (implicit deny). Access exists only through attached policies, so `s3-user-allowed` can use S3 while `s3-user-denied` cannot.

Cleanup (avoid charges): terminate EC2 instances, delete the bucket's objects and the bucket, delete the VPC (after removing IGW and subnets), and delete the IAM users and role.

Assignment 9: AWS Lambda + DynamoDB + S3

Use the Python 3.12 runtime for all functions in one region (e.g. ap-south-1).

One-time setup: DynamoDB -> Create table -> Name `Students`, Partition key `PRN` (String). For Q3 to Q8 add `AmazonDynamoDBFullAccess` to the Lambda execution role (Configuration -> Permissions -> role name -> Add permissions -> Attach policies).

To create each function: Lambda -> Create function -> Author from scratch -> name -> Python 3.12 -> Create. Paste the code, click Deploy, then use Test with the event shown.

Q1. Add two numbers (add-numbers)

```
def lambda_handler(event, context):
    a = event['num1']
    b = event['num2']
    result = a + b
    print(f"{a} + {b} = {result}")
    return {"statusCode": 200, "sum": result}
```

Test event:

```
{ "num1": 10, "num2": 25 }
```

Output: `{"statusCode": 200, "sum": 35}`

Q2. Accept name, PRN, course and print (print-student)

```
def lambda_handler(event, context):
    name = event['name']
    prn = event['prn']
    course = event['course']
    print(f"Name: {name}")
    print(f"PRN: {prn}")
    print(f"Course: {course}")
    return {"statusCode": 200, "message": "Printed successfully"}
```

Test event:

```
{ "name": "Amit", "prn": "123456", "course": "DevOps" }
```

Check the printed lines under Execution results -> Log output.

Q3. Insert student into DynamoDB (insert-student)

```
import boto3

dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('Students')

def lambda_handler(event, context):
    table.put_item(Item={
        'PRN': event['prn'],
        'name': event['name'],
        'course': event['course']
    })
    return {"statusCode": 200, "message": "Student inserted"}
```

Test event:

```
{ "prn": "123456", "name": "Amit", "course": "DevOps" }
```

Verify: DynamoDB -> Tables -> Students -> Explore table items.

Q4. Read student by PRN (read-student)

```
import boto3

dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('Students')

def lambda_handler(event, context):
    response = table.get_item(Key={'PRN': event['prn']})
    item = response.get('Item')
    if item:
        print(item)
        return {"statusCode": 200, "student": item}
    return {"statusCode": 404, "message": "Student not found"}
```

Test event:

```
{ "prn": "123456" }
```

Q5. Update a student's course (update-student-course)

```
import boto3

dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('Students')

def lambda_handler(event, context):
    response = table.update_item(
        Key={'PRN': event['prn']},
        UpdateExpression='SET course = :c',
        ExpressionAttributeValues={':c': event['new_course']},
        ReturnValues='UPDATED_NEW'
    )
    return {"statusCode": 200, "updated": response['Attributes']}
```

Test event:

```
{ "prn": "123456", "new_course": "Cloud Computing" }
```

Q6. Delete a student using PRN (delete-student)

```
import boto3

dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('Students')

def lambda_handler(event, context):
    table.delete_item(Key={'PRN': event['prn']})
    return {"statusCode": 200, "message": f"Student {event['prn']} deleted"}
```

Test event:

```
{ "prn": "123456" }
```

Q7. Scan all student records (scan-students)

```
import boto3

dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('Students')

def lambda_handler(event, context):
    response = table.scan()
    items = response['Items']

    # handle pagination for large tables
    while 'LastEvaluatedKey' in response:
        response = table.scan(ExclusiveStartKey=response['LastEvaluatedKey'])
        items.extend(response['Items'])

    print(items)
    return {"statusCode": 200, "count": len(items), "students": items}
```

Test event: `{}`. Insert a few students first (Q3) so the scan returns data.

Q8. Check PRN exists before inserting (safe-insert-student)

```
import boto3

dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('Students')

def lambda_handler(event, context):
    prn = event['prn']

    existing = table.get_item(Key={'PRN': prn})
    if 'Item' in existing:
        return {"statusCode": 409, "message": f"PRN {prn} already exists"}

    table.put_item(Item={
        'PRN': prn,
        'name': event['name'],
        'course': event['course']
    })
    return {"statusCode": 200, "message": "Student inserted"}
```

Test event:

```
{ "prn": "123456", "name": "Amit", "course": "DevOps" }
```

Run it twice. The first run inserts; the second returns `409 ... already exists`.

Q9. S3 trigger: print bucket name and file name

1. Create the bucket: S3 -> Create bucket, e.g. `my-lambda-trigger-bucket-001` (same region as the Lambda).

2. Create function `s3-upload-logger`**:**

```
import urllib.parse

def lambda_handler(event, context):
    record = event['Records'][0]
    bucket = record['s3']['bucket']['name']
    key = urllib.parse.unquote_plus(record['s3']['object']['key'])
    print(f"Bucket name: {bucket}")
    print(f"File name: {key}")
    return {"statusCode": 200}
```

3. Add the trigger: Function page -> Add trigger -> S3 -> select the bucket -> Event type: All object create events -> tick the recursive invocation acknowledgment -> Add.

4. Test: Upload any file to the bucket, then Lambda -> Monitor -> View CloudWatch logs -> latest log stream:

```
Bucket name: my-lambda-trigger-bucket-001
File name: sample.txt
```

Q10. S3 trigger: print the size of the uploaded file

Function name: `s3-file-size`

```
import urllib.parse

def lambda_handler(event, context):
    record = event['Records'][0]
    bucket = record['s3']['bucket']['name']
    key = urllib.parse.unquote_plus(record['s3']['object']['key'])
    size = record['s3']['object']['size']
    print(f"File: {key} in bucket: {bucket}")
    print(f"Size: {size} bytes")
    return {"statusCode": 200}
```

The S3 event already carries the size, so no extra S3 call or permission is needed.

Trigger: Add an S3 trigger on a bucket. S3 rejects a second overlapping trigger on the same bucket and event type, so either use a second bucket, delete the Q9 trigger first, or run both from one function.

Test: Upload a file and check CloudWatch logs:

```
File: sample.txt in bucket: my-lambda-trigger-bucket-001
Size: 1024 bytes
```

Console test without uploading: use the s3-put event template under Test and set `"size": 1024`.

Cleanup: delete the Lambda functions, the `Students` table, and the S3 buckets.

Assignment 10: Terraform + Ansible

Prerequisites: Terraform installed and AWS credentials configured (`aws configure` or an IAM role). Use your own PRN and name where they appear.

Q1. Ubuntu EC2 with Web Server (Terraform)

Create a folder `tf-ec2` with this file.

main.tf

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "ap-south-1"
}

# Latest Ubuntu 22.04 AMI (Canonical)
data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"]

  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*"]
  }
}

resource "aws_security_group" "web_sg" {
  name        = "web-sg"
  description = "Allow SSH and HTTP"

  ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_instance" "web" {
  ami                    = data.aws_ami.ubuntu.id
  instance_type          = "t3.micro"
  key_name               = "basic-key"   # existing key pair (Assignment 8)
  vpc_security_group_ids = [aws_security_group.web_sg.id]

  user_data = <<-EOF
    #!/bin/bash
    apt-get update -y
```

```
    apt-get install -y nginx
    cat > /var/www/html/index.html <<HTML
    <html>
      <body>
        <h1>PRN: 123456789</h1>
        <h2>Name: Your Name</h2>
      </body>
    </html>
    HTML
    systemctl enable nginx
    systemctl restart nginx
  EOF

  tags = {
    Name = "terraform-web-server"
  }
}

output "public_ip" {
  value = aws_instance.web.public_ip
}
```

Run:

```
terraform init
terraform plan
terraform apply -auto-approve
```

Verify: open http://<public_ip> (shown in the output). Wait 1 to 2 minutes after apply for user_data to finish. The page displays your PRN and name. Cleanup with `terraform destroy`.

Q2. Create IAM user cdac and delete existing user ditiss

Use a separate folder `tf-iam`.

Create cdac - main.tf

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "ap-south-1"
}

resource "aws_iam_user" "cdac" {
  name = "cdac"
}

terraform init
terraform apply -auto-approve
```

Delete the existing user ditiss: Terraform can only delete resources in its state, and `ditiss` was created outside Terraform, so import it first, then destroy it.

Step 1. Add a resource block:

```
resource "aws_iam_user" "ditiss" {
  name          = "ditiss"
  force_destroy = true     # also deletes access keys, login profile, MFA, etc.
}
```

Step 2. Import the existing user into the state:

```
terraform import aws_iam_user.ditiss ditiss
terraform plan
terraform apply -auto-approve
```

Step 3. Delete it by removing the block from main.tf and applying:

```
terraform apply -auto-approve
# plan shows: aws_iam_user.ditiss will be destroyed
```

Or in one command without editing the file:

```
terraform destroy -target=aws_iam_user.ditiss -auto-approve
```

The final main.tf contains only `cdac`.

Verify: IAM -> Users shows `cdac` and no `ditiss`, or run `aws iam list-users`.

Note: `force_destroy = true` is needed if `ditiss` has an access key, console password, or MFA device. Policies attached directly to the user must be detached first, otherwise the delete fails with DeleteConflict.

Q3. Ansible in Client-Server Mode

Ansible is agentless. The server is the control node where Ansible is installed; the client is the managed node reached over SSH. Use two Ubuntu machines (EC2 instances in the same VPC/security group with SSH allowed between them).

On the Ansible server

1. Install Ansible:

```
sudo apt update
sudo apt install -y software-properties-common
sudo add-apt-repository --yes --update ppa:ansible/ansible
sudo apt install -y ansible
ansible --version
```

2. Set up passwordless SSH to the client:

```
ssh-keygen -t rsa -b 4096            # press Enter for all prompts
ssh-copy-id ubuntu@<client-private-ip>
ssh ubuntu@<client-private-ip>       # should log in without a password
exit
```

On EC2, where password login is disabled, append the server's `~/.ssh/id_rsa.pub` to `~/.ssh/authorized_keys` on the client instead.

3. Create the inventory (`~/ansible/inventory.ini`):

```
mkdir ~/ansible && cd ~/ansible

[clients]
client1 ansible_host=<client-private-ip> ansible_user=ubuntu
```

4. Test the connection:

```
ansible -i inventory.ini clients -m ping
```

Expected: `client1 | SUCCESS => { "ping": "pong" }`

5. Create the shell script (`hello.sh`):

```
#!/bin/bash
echo "Hello DITISS"

chmod +x hello.sh
```

Execute the script on the client

Option A: ad-hoc command (the `script` module copies and runs the script on the client):

```
ansible -i inventory.ini clients -m script -a "./hello.sh"
```

Option B: playbook (`run_script.yml`):

```
---
- name: Run shell script on client
  hosts: clients
  tasks:
```

```
    - name: Execute hello.sh on the client
      script: ./hello.sh
      register: result

    - name: Show output
      debug:
        msg: "{{ result.stdout }}"
```

```
ansible-playbook -i inventory.ini run_script.yml
```

Expected output:

```
TASK [Show output] ***
ok: [client1] => {
    "msg": "Hello DITISS\n"
}
```

The script runs on the client machine. It is copied there temporarily, executed, and the output is returned to the Ansible server.